

CHAPTER 5 USER INTERFACE DESIGN, FIRST PROTOTYPE WORK, AND DELIVERABLE I PREPARATION

INTRODUCTION

Week 5 is an important transition point in the course. In the earlier weeks, students learned the big picture of system development, project foundations, Agile Software Development Life Cycle, or Agile SDLC, requirements, user stories, and Unified Modeling Language, or UML, modeling. By Week 5, the project must begin to look more real to a user. This is why the course outline places User Interface Design as the main Week 5 lecture topic, together with Project Presentation, Deliverable I, and the Deliverable I Report. In other words, this is the week where the system starts becoming visible, demonstrable, and easier to evaluate.

The same outline also makes it clear that the course is project-based, that C# is the language used, and that applications in the course involve user interfaces, event-driven programming, and database connections. Week 5 is therefore the natural point where the earlier analysis and modeling work begin to turn into actual screens, controls, interaction flow, and the first meaningful prototype material.

5.1 WHAT IS A USER INTERFACE?

A User Interface, or UI, is the part of the system that users see and interact with directly. It includes screens, forms, buttons, text boxes, menus, labels, messages, icons, and navigation elements. If a database stores the information and the program logic controls how the system behaves, the user interface is the visible layer through which people actually use the system.

Students sometimes think user interface design is mainly about making the system look attractive. Appearance matters, but it is only one part of the job. A good interface must first help the user complete real tasks clearly, correctly, and with as little confusion as possible.

For example, in an advising appointment system, the interface is not just the color of the page or the shape of a button. It is the full experience of how the student finds a time slot, enters information, confirms the request, and understands the result. If that experience is confusing, the system is weak even if the database and code are technically correct. A well-designed interface makes the difference between a system that users love and one they avoid.

5.2 WHY USER INTERFACE DESIGN MATTERS IN SYSTEM DEVELOPMENT

User interface design matters because the interface is where the user experiences the system. Many students build projects that are logically correct in the background but frustrating on the front end. When that happens, users often judge the whole system as poor, even if the internal code structure is acceptable.

A badly designed interface can create several problems:

-  It can confuse users.
-  It can hide important actions.
-  It can increase input errors.
-  It can slow down common tasks.
-  It can make the system feel unreliable or unfinished.

A well designed interface does the opposite. It guides the user naturally, reduces mistakes, supports the business process, and makes the project easier to demonstrate in presentations.

This is exactly why Week 5 appears where it does in the course. By now, students already know the project goals, stakeholders, requirements, user stories, and UML models. The interface should now emerge from those earlier decisions instead of being invented randomly.

5.3 USER INTERFACE DESIGN IS NOT DECORATION

One of the most important things students must understand in Week 5 is that user interface design is not decoration. It is not a final cosmetic layer added after all the important technical work is done. It is part of system design itself.

A form, for example, is a design decision. The number of fields on the form, the order of those fields, the labels used, the validation messages, and the placement of the submit button all affect the user's success. A screen that asks for too much information, uses poor wording, or hides important actions is not just unattractive. It is badly designed.

That means user interface design belongs to the same serious design process as requirements analysis, user stories, and modeling. The course outline places it in Week 5 precisely because it builds directly on the work of Weeks 2, 3, and 4.

5.4 PRINCIPLES OF GOOD USER INTERFACE DESIGN

There are several basic principles that students should follow.

Clarity The interface should make its purpose obvious. A student opening a screen should quickly understand what the screen is for and what action should happen next.

Simplicity A screen should include what is necessary, not everything imaginable. Students often overload their first screens with too many fields, too many buttons, or too much text. A simple interface is usually easier to use and easier to test.

Consistency Similar actions should look and behave in similar ways across the system. If one screen uses "Save" and another screen uses "Submit" for the same action, users may hesitate. If one form places the confirm button at the bottom right and another hides it somewhere else without reason, consistency is lost.

Feedback The system should respond clearly to user actions. If a user clicks a button, submits a form, or enters invalid data, the interface should show what happened. Silence creates uncertainty.

Error prevention A strong interface helps stop mistakes before they happen. Good label wording, input restrictions, sensible defaults, and validation can all reduce user errors.

Efficiency The interface should help users complete tasks without unnecessary steps. Every extra click, unclear field, or repeated action increases frustration.

User-centered thinking A screen should be designed for the user's real task, not for what looks impressive to the developer. This principle ties directly back to the stakeholder analysis and social responsibility topics from Week 2.

5.5 CONNECTING WEEK 5 TO EARLIER WEEKS

Week 5 should not stand alone. It must grow directly from the work students already completed.

From Week 2, students have requirements, stakeholder thinking, constraints, and early design notes. From Week 3, they have user stories, acceptance criteria, and iteration planning. From Week 4, they have use case diagrams, class diagrams, and state transition thinking.

Now, in Week 5, those ideas should become visible interface decisions.

For example:

- ✚ A user story such as “As a student, I want to request an advising appointment online so that I can avoid back-and-forth email messages” naturally suggests a request form screen.
- ✚ A class diagram that includes Student, Appointment, and Advisor helps students decide what data fields may need to appear on forms.
- ✚ A state transition diagram for an Appointment object helps the student understand what statuses may need to be shown in the interface, such as Requested, Approved, Cancelled, or Completed.

This is why Week 5 is such an important bridge between analysis and implementation.

5.6 MAIN TYPES OF USER INTERFACE ELEMENTS

- ✚ Students should become familiar with common interface controls and their purposes.
- ✚ **Label:** A label displays text that helps the user understand a field, screen, or section. Labels should be clear and direct.
- ✚ **Text box:** A text box allows the user to enter text. It is useful for names, reasons, descriptions, and other typed information.
- ✚ **Button:** A button triggers an action such as Save, Submit, Search, Update, or Cancel.
- ✚ **Combo box:** A combo box allows the user to choose from a list of predefined options while keeping the screen compact.
- ✚ **List box:** A list box displays a list of items, often when several choices need to remain visible.
- ✚ **Check box:** A check box allows the user to select or deselect an option.
- ✚ **Radio button:** Radio buttons are useful when the user must choose exactly one option from a small group.
- ✚ **Menu:** A menu helps organize actions and navigation in a larger application.
- ✚ **Message box:** A message box shows important feedback or warnings to the user.

Students should learn not only what these controls are called, but why one control is better than another in a given situation. Choosing the right control is part of good design.

5.7 SCREEN DESIGN AND LAYOUT

A good interface is not just a collection of controls. The controls must be arranged logically.

A strong layout usually follows the user's natural reading and task order. Information that belongs together should appear together. Important actions should be easy to find. Labels should be close to the fields they describe. Spacing should help the eye, not create clutter.

A common beginner mistake is to place controls randomly on a form until the window looks full. That is not design. Good layout comes from asking:

- ✚ What is the user trying to do first?
- ✚ What information do they need to see?
- ✚ What should be entered next?
- ✚ What action ends the task?

If those questions are answered well, the screen becomes easier to understand.

5.8 NAVIGATION DESIGN

The interface should help users move through the system logically.

In small student projects, navigation may be simple. A main form may open one or two smaller forms. In a larger project, users may need a menu, dashboard, or clear set of navigation buttons.

Good navigation should answer these questions:

- ✚ Where am I?
- ✚ What can I do here?
- ✚ How do I return or continue?
- ✚ What is the main action on this screen?

Confusing navigation makes even a small project feel weak.

5.9 Input design and data entry

Data entry is one of the most important parts of user interface design because many systems depend on correct input.

Students should think carefully about:

- ✚ Which fields are required,
- ✚ Which fields are optional,
- ✚ What format the input should have,
- ✚ How long the input can be,
- ✚ What should happen when the input is wrong,
- ✚ And how the user will understand the error.

For example, in an appointment request form, a student may need to enter student number, preferred date, preferred time, reason for the appointment, and possibly advisor selection. If these fields are badly arranged or poorly labeled, the user may enter wrong or incomplete data.

5.10 VALIDATION IN USER INTERFACE DESIGN

Validation means checking whether user input is acceptable before the system continues.

Validation is essential because systems must not trust raw input automatically. Users may leave a field empty, type letters where a number is expected, choose an impossible date, or submit incomplete information.

A strong interface should perform basic validation and give clear, respectful feedback. Bad validation message: “Error.” Better validation message: “Please enter a valid student number before submitting the request.”

Students should understand that validation is part of both usability and data quality.

5.11 ERROR MESSAGES AND USER FEEDBACK

When users do something wrong, the system should not punish them with vague or hostile messages.

A good error message should:

- ✚ Say what went wrong,
- ✚ Help the user understand why,
- ✚ And tell them what to do next.

For example:

- ✚ “The appointment date cannot be in the past.”
- ✚ “Please select an advisor before continuing.”
- ✚ “This email field cannot be left blank.”

Good feedback is just as important when things go right:

- ✚ “Appointment request submitted successfully.”
- ✚ “Changes saved.”
- ✚ “Record updated.”

These small details improve trust in the system.

5.12 ACCESSIBILITY AND INCLUSIVE INTERFACE DESIGN

Earlier in the course, students already discussed social responsibility. Week 5 should continue that thinking in interface design.

An interface should try to remain understandable and usable for different users. That includes using readable text, clear labels, reasonable contrast, logical structure, and avoiding unnecessary complexity.

Students do not need to become accessibility experts in one week, but they should begin asking:

- ✚ Would a first-time user understand this screen?
- ✚ Are the instructions clear?
- ✚ Are labels meaningful?
- ✚ Is the system forcing confusing behavior on the user?

An interface that looks fine to the developer may still be difficult for real users.

5.13 USER-CENTERED DESIGN

A user-centered design approach means students design screens around the actual needs and workflow of the user rather than around whatever is easiest to code.

This is where stakeholder thinking becomes important again. The student user and the advisor user may need very different screens. A student may need a simple request form and a clear appointment list. An advisor may need a dashboard with approval actions and schedule controls.

A strong project does not assume one screen fits everyone.

5.14 PROTOTYPING IN WEEK 5

A prototype is an early version of the system that shows how the system may work. In Week 5, students do not need a complete final system. They need a meaningful first version that demonstrates progress.

A prototype can be a sketch, a wireframe, a clickable form, a partial Windows Forms interface, or a small working piece of the application. The goal of the prototype is not perfection. The goal is to make the design visible, testable, and discussable.

The course outline also says students should move from initial analysis and requirements toward designing a prototype and implementing core parts to demonstrate validity. That makes Week 5 an ideal time to begin showing visible prototype work.

5.15 WIREFRAMES AND MOCKUPS

Before writing too much code, students often benefit from making a quick wireframe.

A wireframe is a simple visual plan of a screen. It shows placement of elements such as labels, text boxes, buttons, and menus, but usually without full styling. A mockup is often a more polished version of the same screen idea.

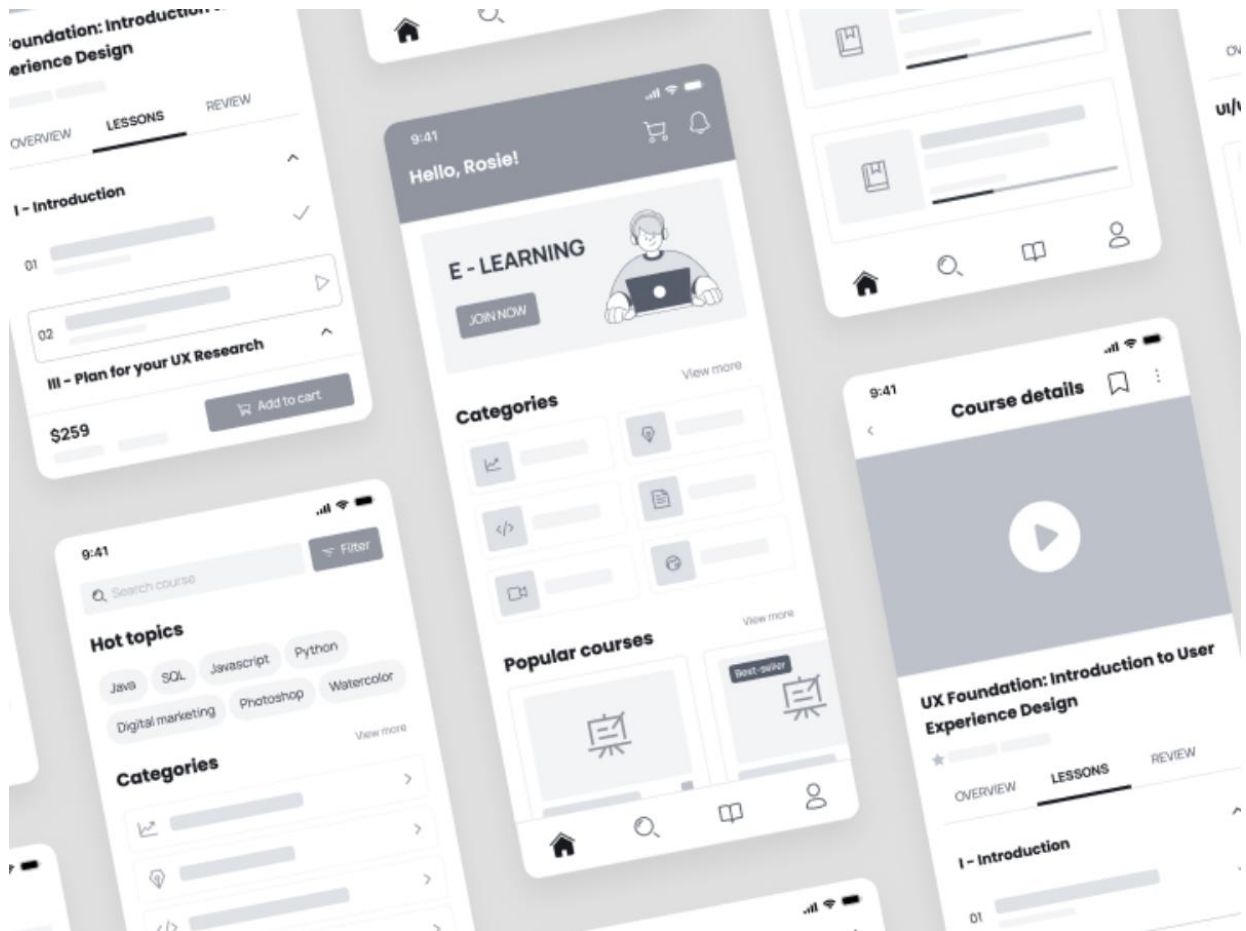
Even a simple wireframe can help answer useful questions:

- ✚ Is this screen overloaded?
- ✚ Are the fields in the right order?
- ✚ Does the main action stand out?
- ✚ Is anything missing?

Students who sketch first often make better interface decisions.

FIGURE 5.1: EXAMPLE OF A CLEAN APPOINTMENT REQUEST FORM WIREFRAME/MOCKUP

(This shows a simple, professional layout that students can use as a model for their own project.)



visily.ai

5.16 TURNING USER STORIES INTO SCREENS

One of the best ways to build Week 5 correctly is to map user stories to screens.

For example: User story: “As a student, I want to request an advising appointment online so that I can avoid email delays.” Possible interface screen: Appointment Request Form Possible controls: student number text box, date selector, time selector, reason text box, submit button, cancel button.

Another example: User story: “As an advisor, I want to view pending requests so that I can approve or reject them quickly.” Possible interface screen: Advisor Request Dashboard Possible controls: request list, approve button, reject button, status label, filter options.

This method keeps interface design grounded in earlier project work.

5.17 EVENT-DRIVEN PROGRAMMING IN WEEK 5

The course outline says the applications in this course involve event-driven programming, user interfaces, and database connections. Week 5 is where that begins to feel real.

Event-driven programming means the program waits for user actions and reacts to them. A button click, menu selection, form submission, or text change can trigger code.

This is why C# is useful in this course. In a Windows Forms interface, students can place controls on a form and write code that responds to events.

5.18 BEGINNING SERIOUS C# PRACTICAL WORK IN CHAPTER 5

Chapter 1 introduced C# lightly. Week 5 is where C# should become more practical.

Students should now begin working with forms, labels, text boxes, buttons, basic event handlers, simple validation, and basic interface navigation. For example, a button click event in C# may be used to check whether a required field is empty, display a message, or open another form.

At this stage, students do not need a giant codebase. They need working pieces that reflect their user stories and support Deliverable I.

5.19 PRACTICAL C# FOR USER INTERFACE DESIGN

Week 5 is the right moment to begin using C# in a more practical way. In Chapter 1, students were introduced to C# lightly. That was appropriate because Week 1 was mainly about course foundations, project setup, and Agile Software Development Life Cycle. By Week 5, however, students are working on user interface design, early prototypes, and Deliverable I. At this point, C# should begin to move from theory into visible interface work.

The goal in this section is not to turn the course into a separate programming course. The goal is to show students how their interface ideas can become real working screens. In other words, this section teaches C# as a tool for system development.

5.19.1 WHY C# FITS USER INTERFACE DESIGN IN THIS COURSE

C# is well suited to this course because it works well with Windows Forms, which is a Microsoft framework for creating graphical desktop applications. A graphical desktop application is a program with visible windows, buttons, text boxes, labels, and menus. This is useful in a system development course because students often need to create forms and screens before they build a full production system.

C# also fits naturally with event-driven programming. Event-driven programming means the program waits for the user to do something, such as click a button or type text, and then responds. This is exactly how most interface-based systems behave.

For example:

-  when a user clicks Submit, the system checks the input
-  when a user clicks Clear, the fields are emptied
-  when a user clicks Next, another form opens
-  when a user enters invalid data, a message appears

This style of programming connects directly to Week 5, because user interface design is not complete until the interface can react to the user.

5.19.2 A SMALL PRACTICAL EXAMPLE: APPOINTMENT REQUEST FORM

To keep the example consistent with the rest of the course, we will use a simple advising appointment request form.

Imagine that the student is designing a first prototype for an advising system. A very simple form might contain:

- ✚ a title label
- ✚ a text box for Student Number
- ✚ a text box for Reason
- ✚ a button called Submit
- ✚ a button called Clear

This is enough to teach the first practical C# ideas:

- ✚ reading values from controls
- ✚ checking whether fields are empty
- ✚ displaying a message box
- ✚ clearing input fields

5.19.3 CREATING THE WINDOWS FORMS PROJECT

Students should begin with these steps:

1. Open Visual Studio.
2. Click Create a new project.
3. Select Windows Forms App using C#.
4. Name the project something simple, such as AdvisingRequestPrototype.
5. Click Create.

When the form opens, students should add these controls:

- ✚ Label
- ✚ TextBox for student number
- ✚ TextBox for reason
- ✚ Button for submit
- ✚ Button for clear

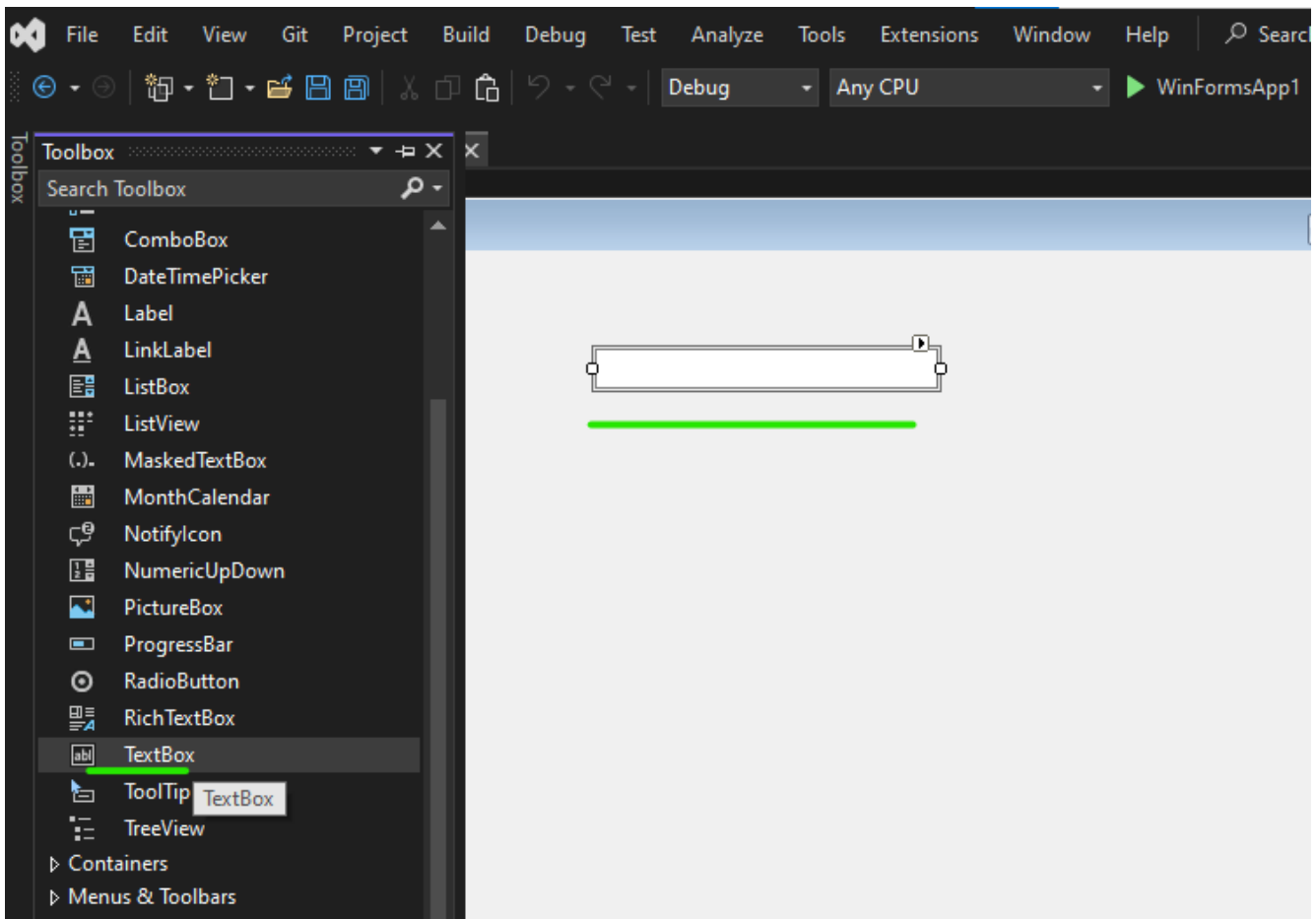
Students should also rename the controls in a meaningful way. For example:

- ✚ lblTitle
- ✚ txtStudentNumber
- ✚ txtReason
- ✚ btnSubmit
- ✚ btnClear

This is important because meaningful control names make code easier to read later.

FIGURE 5.2 – VISUAL STUDIO WINDOWS FORMS DESIGNER WITH TOOLBOX

(This is exactly what students see when they start a new Windows Forms project in Week 5.)



media.geeksforgeeks.org

5.19.4 DESIGNING THE FORM BEFORE CODING

Before writing code, students should think about the interface itself.

A simple good layout might be:

- ✚ title label at the top
- ✚ student number label and text box under it
- ✚ reason label and text box below that
- ✚ submit and clear buttons at the bottom

This design matters because good coding cannot rescue a badly organized form. Week 5 is still a design week first, even when coding begins.

Students should ask:

- ✚ Is the main action obvious?
- ✚ Are the labels clear?
- ✚ Is the order of fields logical?
- ✚ Would a first-time user understand what to do?

5.19.5 WRITING THE SUBMIT BUTTON CODE

Now students can double-click the Submit button in the form designer. Visual Studio will generate a button click method.

Below is a simple example with a comment on every single line.

C#

```
private void btnSubmit_Click(object sender, EventArgs e) // This method runs automatically when the user clicks
the Submit button. sender is the control that triggered the event, and e contains event information.
{
    string studentNumber = txtStudentNumber.Text; // This line reads the text currently written in the student
number text box and stores it in a variable called studentNumber.

    string reason = txtReason.Text; // This line reads the text currently written in the reason text box and stores it in a
variable called reason.

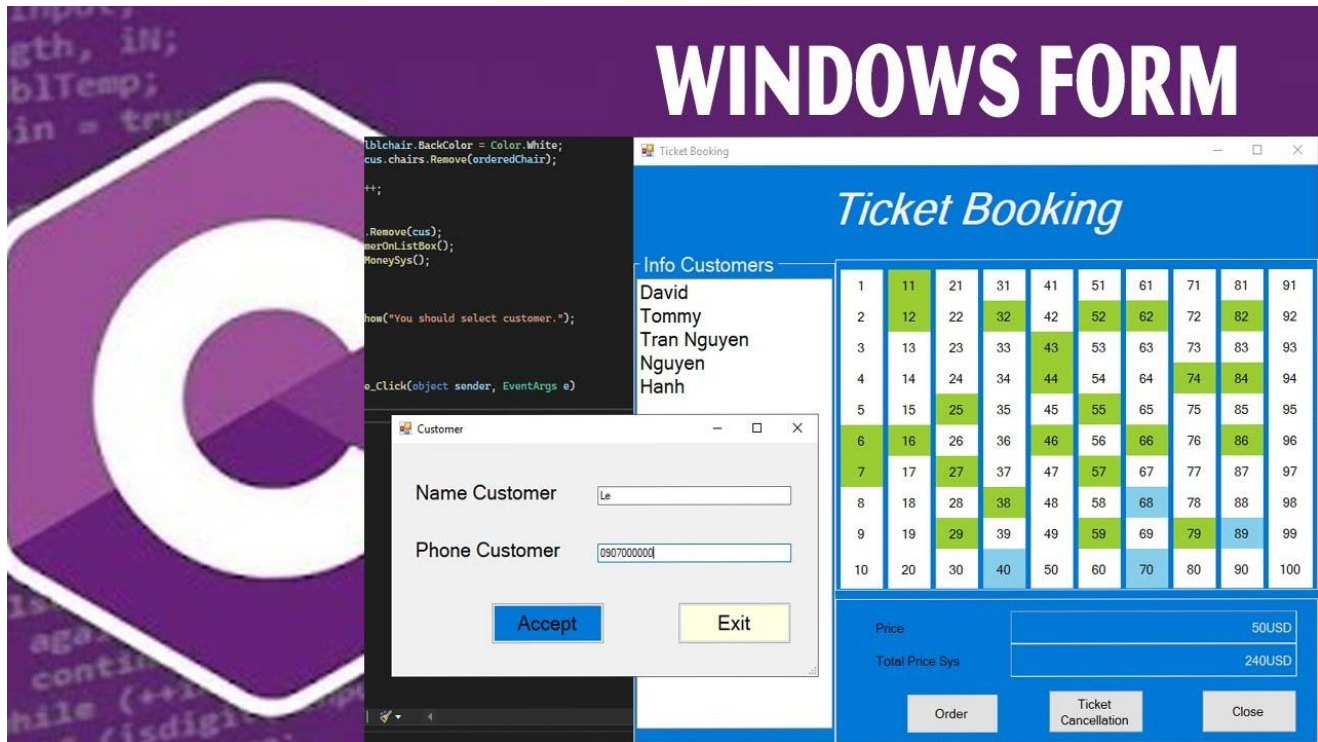
    if (studentNumber == "" || reason == "") // This line checks whether either field is empty. == means "is equal to"
and || means "or".
    {
        MessageBox.Show("Please complete all required fields before submitting.", "Missing Information"); // This line
opens a popup message box telling the user that required fields are missing. The first part is the message, and the
second part is the popup title.

        return; // This line stops the method immediately so the rest of the code does not continue running when the
input is incomplete.
    }

    MessageBox.Show("Appointment request submitted successfully.", "Success"); // This line shows a success
message if both fields were completed correctly.
}
```

FIGURE 5.3 – EXAMPLE OF A COMPLETED SIMPLE C# WINDOWS FORMS APPOINTMENT REQUEST SCREEN

(This shows what the working form looks like after the code above is added.)



i.ytimg.com

5.19.6 BREAKING DOWN THE VALIDATION LOGIC

Students often need more explanation for the if statement, because this is where the code begins to make decisions.

Look at this line again:

C#

```
if (studentNumber == "" || reason == "")
```

Here is the meaning in plain language:

- ✚ if means the program is asking a question
- ✚ studentNumber == "" means "is the student number field empty?"
- ✚ reason == "" means "is the reason field empty?"
- ✚ || means "or"

So the full line means: If the student number field is empty, or the reason field is empty, then show the missing information message.

This is a good beginner example because it connects directly to real interface behavior.

5.19.7 WRITING THE CLEAR BUTTON CODE

Now let us add another small but useful feature. When the user clicks Clear, the form should empty the text boxes.

Double-click the Clear button and use code like this:

C#

```
private void btnClear_Click(object sender, EventArgs e) // This method runs automatically when the user clicks the
Clear button.

{

    txtStudentNumber.Text = ""; // This line replaces the content of the student number text box with an empty
string, which clears the field.

    txtReason.Text = ""; // This line replaces the content of the reason text box with an empty string, which clears the
reason field.

    txtStudentNumber.Focus(); // This line places the typing cursor back inside the student number text box so the
user can start entering data again immediately.

}
```

WHY THIS CODE MATTERS

This small method teaches students that interface design includes convenience, not only correctness. The form is easier to use because the fields can be cleared quickly, the cursor returns to the first field, and the user does not need to click manually to start again. This is a good example of how coding supports usability.

5.19.8 IMPROVING THE FORM WITH BETTER VALIDATION

The first version checks only whether fields are empty. Students can improve the prototype further.

For example, if the student number should contain only digits, they can add another validation step:

C#

```
private void btnSubmit_Click(object sender, EventArgs e) // This method runs when the Submit button is clicked.

{

    string studentNumber = txtStudentNumber.Text; // Reads the student number from the text box.
```

```

string reason = txtReason.Text; // Reads the reason from the text box.

if (studentNumber == "" || reason == "") // Checks whether one of the required fields is empty.
{
    MessageBox.Show("Please complete all required fields before submitting.", "Missing Information"); // Shows a
warning message.

    return; // Stops the method here if the validation fails.
}

if (!int.TryParse(studentNumber, out _)) // This line checks whether the student number can be converted into an
integer. The ! means "not", so this condition becomes true if the value is not a valid whole number.
{
    MessageBox.Show("Student number must contain digits only.", "Invalid Student Number"); // Shows a
message explaining the specific problem.

    return; // Stops the method if the student number is invalid.
}

MessageBox.Show("Appointment request submitted successfully.", "Success"); // Shows success if all validation
checks pass.
}

```

WHAT STUDENTS SHOULD UNDERSTAND HERE

This example introduces `int.TryParse()`, which checks whether text can be turned into an integer. An integer is a whole number. The important lesson is not the exact method name. The important lesson is this: A good interface checks that data is not only present, but also valid. That is why validation belongs to both design and programming.

5.19.9 OPENING ANOTHER FORM

Many systems need more than one screen. A student may want one form for appointment requests and another form for viewing appointments.

Suppose there is a second form called `FormViewAppointments`. A button on the first form could open it like this:

C#

```
private void btnViewAppointments_Click(object sender, EventArgs e) // This method runs when the View
Appointments button is clicked.
{
    FormViewAppointments viewForm = new FormViewAppointments(); // This line creates a new object from the
FormViewAppointments class and stores it in a variable called viewForm.

    viewForm.Show(); // This line opens the second form on the screen.
}
```

WHY THIS MATTERS

This teaches an important system development idea: a project is often made of several connected screens, not one single giant window. It also helps students begin thinking about navigation, which is a core part of interface design.

5.19.10 CONNECTING INTERFACE CONTROLS TO USER STORIES

Students should never see these C# examples as isolated code fragments. Each one should connect back to earlier project work.

For example: User story: As a student, I want to request an advising appointment so that I can avoid email delays. Interface design: student number text box, reason text box, submit button. C# logic: read the values, validate them, show success or error messages.

This is the exact teaching flow the course needs: requirements and user stories first, then interface design, then C# behavior.

5.19.11 WHAT STUDENTS ARE NOT EXPECTED TO MASTER YET

Week 5 is an important practical milestone, but students should not feel they must already build a full professional application.

At this stage, they are not expected to master advanced object-oriented architecture, full database integration, complex exception handling, polished multi-form system design, or production-level security.

Instead, they should focus on:

- ✚ building a clear first interface
- ✚ making buttons work
- ✚ validating basic input
- ✚ demonstrating visible project progress
- ✚ linking interface behavior to requirements and user stories

That is enough for a strong first prototype.

5.19.12 HOW THIS SECTION SUPPORTS DELIVERABLE I

The Week 5 deliverable is not supposed to be the final finished system. It is the first serious milestone. That means this new practical C# section helps students show something meaningful by Week 5.

For example, a student could include in Deliverable I: one working form, labeled fields, one or two buttons, button click events, basic validation, simple message boxes, screenshots and explanation in the report.

That is realistic, visible, and easy to connect to the project story.

5.19.13 PRACTICAL ADVICE FOR TEACHING THIS SECTION

If this section is taught in class, the best approach is not to rush through many controls. It is better to take one small example and explain it well.

A good classroom flow would be:

- ✚ create the form live
- ✚ place the controls
- ✚ rename the controls properly
- ✚ write one Submit event together
- ✚ test the form with both good and bad input
- ✚ add the Clear button
- ✚ discuss how this small prototype connects to the semester project

That way, students do not just copy code. They understand why each part exists.

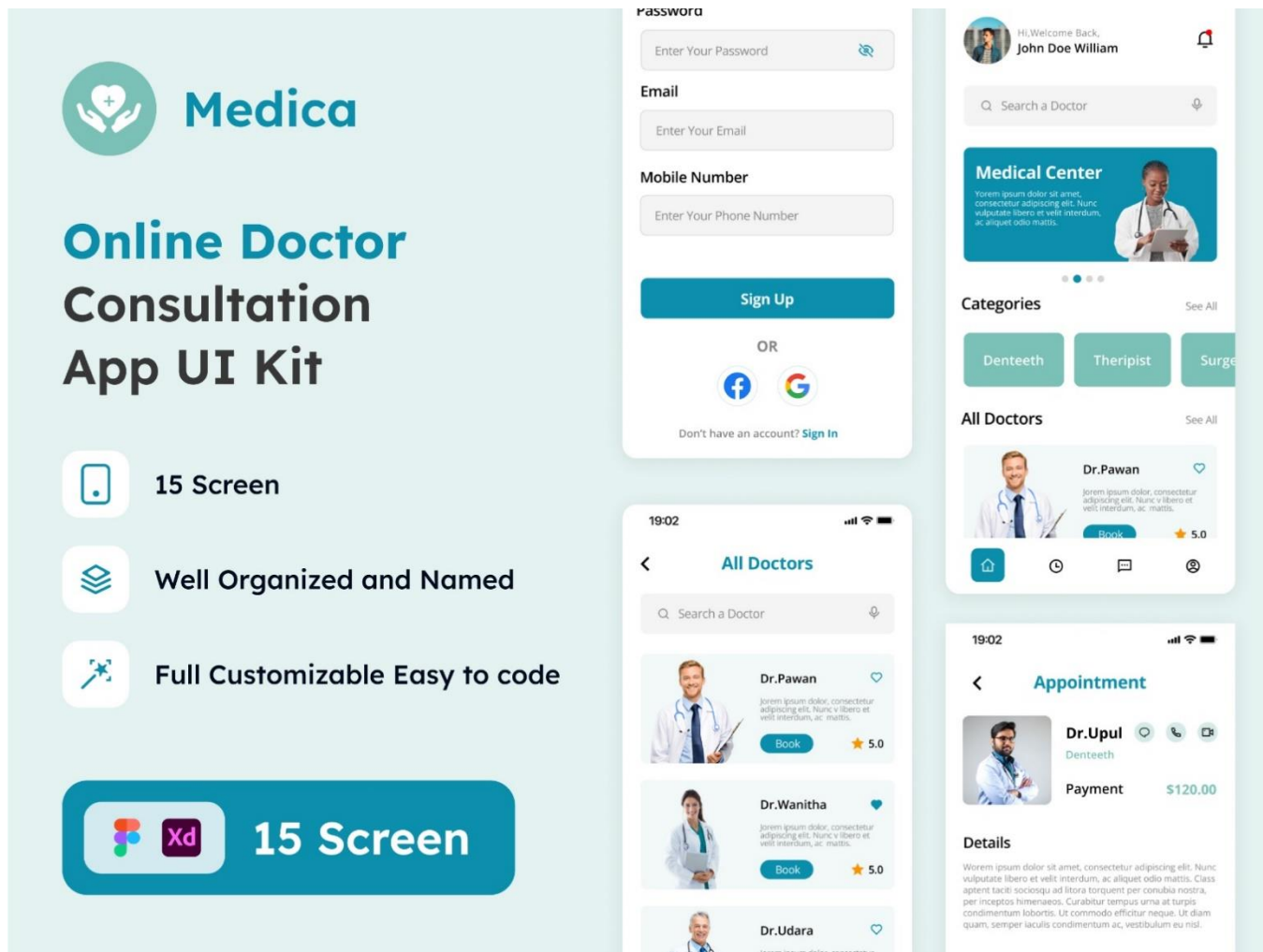
5.19.14 SECTION SUMMARY

This section brings C# into Week 5 in the right way. Students do not study C# here as a separate full programming course. They use it as a practical tool for user interface design. Through a simple appointment request form, they learn how to create controls, respond to button clicks, validate input, clear fields, and open another form.

This is exactly the kind of applied system-development programming that fits the course outline. It turns user stories into working interface behavior and helps students prepare a realistic Deliverable I.

Figure 5.4: Modern example of a polished appointment booking interface

(This shows what a more finished prototype could look like after students apply the principles and code from this chapter.)



s3-alpha.figma.com

5.20 EXAMPLE: A SIMPLE APPOINTMENT REQUEST FORM

Let us imagine a student is building an advising appointment system.

A first screen might include a title label, a student number field, a preferred date field, a reason field, a submit button, and a cancel button.

Good design questions would include:

- ✚ Is the student number field clearly labeled?
- ✚ Does the user understand what date format is needed?
- ✚ Is the reason field optional or required?
- ✚ What happens if the user clicks submit with missing data?
- ✚ What feedback appears after success?

A weak student project often ignores these questions. A stronger project answers them directly in the interface design.

5.21 THE RELATIONSHIP BETWEEN INTERFACE DESIGN AND DATABASE DESIGN

Although database design is a later main week, interface design already hints at database needs.

If a screen collects a student number, appointment time, status, and reason, then students should begin realizing that these values may later become fields in a database table.

That is why Chapter 5 also supports later chapters. A well-designed interface often reveals what data must be stored, what data must be updated, what relationships may exist, and what validation rules may be necessary.

5.22 COMMON MISTAKES STUDENTS MAKE IN WEEK 5

There are several common mistakes in this chapter. One is trying to build too many screens too early. Another is designing from appearance only instead of from user tasks. Another is adding controls without knowing why they are needed. Another is skipping validation and feedback. Another is creating inconsistent naming and layout across forms. Another is ignoring the connection between the interface and the earlier user stories. A final common mistake is trying to make Deliverable I look final instead of making it clear, realistic, and meaningful.

5.23 DELIVERABLE I: WHAT STUDENTS SHOULD SHOW BY WEEK 5

The course outline says Week 5 includes Deliverable I, worth 20% of the project, applying the first round of all phases of Agile SDLC, along with a Deliverable I Report and a Project Presentation.

That means students should not aim for a complete system yet. They should aim for a first solid milestone.

By this point, a student should ideally be able to show a clear project purpose, the main user roles, evidence of requirements and user stories, some UML thinking from Week 4, at least the first meaningful interface screens, and some evidence that the project is moving from design into implementation.

This first deliverable should feel real, but it does not need to feel finished.

5.24 PROJECT PRESENTATION IN WEEK 5

Students should also know how to present their work clearly.

A good Week 5 presentation should explain what problem the project solves, who the users are, what the first interface screens do, how those screens connect to requirements and user stories, what part is complete now, and what will be improved in the next iteration.

Students do not need to make the presentation dramatic. They need to make it understandable.

5.25 DELIVERABLE I REPORT

The report for Deliverable I should not just repeat screenshots without explanation.

A good report should include the project title, a short project purpose, the users, the first iteration scope, key requirements or user stories addressed, interface screenshots or sketches, a brief explanation of what works now, and a short explanation of what comes next.

This report helps the instructor evaluate not only the visible result, but also the student's design thinking.

5.26 WHAT STUDENTS SHOULD KNOW BY THE END OF CHAPTER 5

By the end of this chapter, students should be able to explain what user interface design is and why it matters. They should be able to identify common interface controls, apply principles such as clarity, consistency, simplicity, validation, and feedback, and turn user stories into meaningful screens. They should understand how event-driven programming supports interface interaction in C#, how interface design prepares for later database work, and what kind of prototype progress is expected for Deliverable I.

5.27 CHAPTER SUMMARY

Chapter 5 brings the project into visible form. Students move from requirements, user stories, and UML models into screens, forms, controls, and interaction flow. They learn that user interface design is not decoration. It is a serious design activity that directly affects usability, clarity, error prevention, and project quality.

This chapter also marks the point where C# becomes more practical. Students begin turning interface ideas into real forms and event-driven behavior. Just as important, the chapter prepares students for Week 5 project presentation, Deliverable I, and the Deliverable I Report. In other words, this week is where the project starts to look like a real system instead of only a planned one.

REFERENCES

-  Vanier College, Faculty of Continuing Education. (2025). *Course Outline: System Development (420-940-VA)*.
-  Shelly, G. B., Cashman, T. J., & Rosenblatt, H. J. (2014). *System Analysis and Design* (10th ed.). Course Technology. ISBN: 9781285171340.
-  Kendall, K. E., & Kendall, J. E. (2023). *Systems Analysis and Design*. Pearson Education. ISBN-13: 9780137947850.